



Persistência de Dados

NeDB com Fastify

O Problema com Dados

Sem Persistência



Dados ficam na memória RAM



Reiniciar o servidor → tudo perdido



Impossível compartilhar entre rotas



Nenhuma forma de consultar histórico



Com Persistência



Dados salvos em arquivo no disco



Sobrevivem a reinicializações



Consultáveis com filtros e ordenação



Base para qualquer API real

Toda API real precisa de um banco de dados.

SQL vs NoSQL

Característica	SQL (Relacional)	NoSQL (Não-Relacional)
Estrutura	Tabelas com colunas fixas	Documentos, chave-valor, grafos...
Schema	Rígido (definido antes)	Flexível (pode variar por registro)
Relações	JOINS entre tabelas	Embedding ou referências manuais
Escala	Vertical (mais hardware)	Horizontal (mais servidores)
Exemplos	MySQL, PostgreSQL, SQLite	NeDB, CouchDB, Redis, Cassandra
Uso ideal	Dados estruturados e transacionais	Dados variáveis, alta volume, JSON

Tipos de Bancos NoSQL



Documento

Armazena dados como documentos JSON. Ideal para objetos complexos e aninhados.

Ex: NeDB, CouchDB



Chave-Valor

Pares simples de chave e valor. Muito rápido, ideal para cache e sessões.

Ex: Redis, DynamoDB



Grafo

Dados em nós e arestas. Ideal para redes sociais e sistemas de recomendação.

Ex: Neo4j, ArangoDB



Colunar

Organizado por colunas, não linhas. Ótimo para análise de grandes volumes de dados.

Ex: Cassandra, ScyllaDB

Quando Usar NoSQL?



Dados com estrutura variável (cada registro pode ser diferente)



Alto volume de leituras e escritas simultâneas



Dados naturalmente em formato JSON (APIs REST)



Prototipagem rápida sem configuração de banco



Transações financeiras complexas → prefira SQL



Dados com muitas relações e JOINS complexos → prefira SQL

O que é NeDB?

O que é: Banco de dados NoSQL embutido, escrito 100% em Node.js

Formato: Documentos JSON — igual ao que você já usa nas APIs

Instalação: Zero configuração — apenas `npm install @seald-io/nedb`

Armazena: Arquivo `.db` no disco (ou só em memória se preferir)

API: Inspirada no MongoDB — quem aprender NeDB migra fácil

NeDB é o banco de dados perfeito para aprender NoSQL: zero instalação, zero configuração, tudo em Node.js.

Conceitos: SQL → NeDB

SQL

Schema

Tabela

Linha (Row)

Coluna

JOIN



NeDB

Sem schema

Datastore

Documento (JSON)

Campo

Referência manual

Estrutura de um Documento NeDB

```
{
  "_id": "a1b2c3d4e5f6g7h8",
  "titulo": "Inception",
  "diretor": "Christopher Nolan",
  "ano": 2010,
  "genero": "Ficção Científica",
  "nota": 8.8,
  "ativo": true,
  "tags": ["suspense", "sonhos"],
  "criadoEm": "2024-01-15T10:00:00Z"
}
```

Tipos suportados:

String

Texto simples

Number

Inteiro ou decimal

Boolean

true / false

Array

Lista de valores

Date

Data como string ISO

_id

ID único auto-gerado

O NeDB não exige schema — você pode salvar qualquer objeto JSON válido.

Instalação

Nenhum servidor para instalar. Apenas um pacote npm.

```
# Criar o projeto
mkdir projeto-fastify-nedb && cd projeto-fastify-nedb
npm init -y

# Instalar NeDB (fork moderno com Promises)
npm install @seald-io/nedb

# Outras dependências
npm install fastify @fastify/cors dotenv
npm install -D nodemon
```

Por que **@seald-io/nedb** e não **nedb**?

- ✓ O pacote nedb original foi abandonado em 2016
- ✓ @seald-io/nedb é o fork mais ativo e mantido
- ✓ Suporte nativo a Promises e async/await
- ✓ API 100% compatível — mesmos métodos, mesma lógica

Memória vs Arquivo — Onde salvar os dados?

Só em Memória

```
const db = new Datastore();  
// sem filename = memória
```

Comportamento:

- Dados ficam na RAM enquanto o servidor roda
- Ao reiniciar: tudo apagado
- Muito rápido — sem I/O de disco
- Ideal para: testes, cache, sessões temporárias

Em Arquivo (.db)

```
const db = new Datastore({  
  filename: './data/filmes.db',  
  autoload: true,  
});
```

Comportamento:

- Dados gravados em um arquivo .db no disco
- Sobrevive a reinicializações do servidor
- Arquivo criado automaticamente se não existir
- Ideal para: APIs, projetos reais, dados permanentes

Opções do Datastore

Todas as opções disponíveis ao criar um Datastore:

```
const db = new Datastore({
  filename:    './data/filmes.db', // caminho do arquivo
  autoload:    true,               // carrega automaticamente
  timestampData: true,            // adiciona createdAt e updatedAt
});
```

filename	<i>string</i>	Caminho do arquivo .db. Se omitido, o banco fica só em memória.
autoload	<i>boolean</i>	Se true, carrega os dados do arquivo automaticamente ao instanciar. Se false, você precisa chamar db.loadDatabase() manualmente.
timestampData	<i>boolean</i>	Se true, adiciona createdAt e updatedAt em todo documento automaticamente, sem precisar setar no código.
inMemoryOnly	<i>boolean</i>	Força modo memória mesmo que filename seja informado. Útil para testes automatizados.

CRUD — Create (Inserir)

```
const Datastore = require('@seald-io/nedb');
const db = new Datastore({ filename: './data/filmes.db', autoload: true });

// Inserir um documento — retorna o doc com _id gerado
const filme = await db.insertAsync({
  titulo: 'Matrix',
  diretor: 'Wachowski',
  ano: 1999,
  nota: 8.7,
});

console.log(filme._id); // "a1b2c3d4..." (gerado pelo NeDB)
console.log(filme.titulo); // "Matrix"

// Inserir vários de uma vez
const inseridos = await db.insertAsync([
  { titulo: 'Inception', ano: 2010, nota: 8.8 },
  { titulo: 'Interstellar', ano: 2014, nota: 8.6 },
]);
console.log(inseridos.length); // 2
```

CRUD — Read (Buscar)

```
// Buscar todos
const filmes = await db.findAsync({});

// Com filtro exato
const acoes = await db.findAsync({ genero: 'Ação' });

// Com operador de comparação
const bons = await db.findAsync({ nota: { $gte: 8.0 } });

// Buscar apenas um documento
const filme = await db.findOneAsync({ titulo: 'Matrix' });

// Buscar por _id
const porId = await db.findOneAsync({ _id: 'abc123' });

// Ordenar, limitar e paginar
const pagina1 = await db.findAsync({})
  .sort({ nota: -1 }) // decrescente por nota
  .limit(5)           // máximo 5 por página
  .skip(0);           // pula 0 (página 1)

const pagina2 = await db.findAsync({})
```

Filtros e Operadores de Query

Operador	Significado	Exemplo
\$eq	Igual a	{ nota: { \$eq: 8.7 } }
\$ne	Diferente de	{ genero: { \$ne: 'Terror' } }
\$gt	Maior que	{ ano: { \$gt: 2000 } }
\$gte	Maior ou igual	{ nota: { \$gte: 7.0 } }
\$lt	Menor que	{ ano: { \$lt: 1990 } }
\$lte	Menor ou igual	{ nota: { \$lte: 6.0 } }
\$in	Está na lista	{ genero: { \$in: ['Ação', 'Drama'] } }
\$regex	Expressão regular	{ titulo: { \$regex: /matrix/i } }

CRUD — Update (Atualizar)

```
// Atualizar por filtro
await db.updateAsync(
  { titulo: 'Matrix' },    // filtro
  { $set: { nota: 9.0 } }, // o que mudar
  {}                      // opções
);
```

```
// Atualizar por _id
await db.updateAsync(
  { _id: 'abc123' },
  { $set: { genero: 'Ficção Científica' } },
  {}
);
```

```
// Atualizar vários de uma vez
await db.updateAsync(
  { nota: { $lt: 5 } },
  { $set: { ativo: false } },
  { multi: true }
);
```

Operadores de Update

\$set

Define o valor de um campo

```
{ $set: { nota: 9.0, genero: 'Drama' } }
```

\$unset

Remove um campo do documento

```
{ $unset: { genero: true } }
```

\$inc

Incrementa (ou decrementa) um número

```
{ $inc: { nota: 0.5 } }
```

\$push

Adiciona um item a um array

```
{ $push: { tags: 'clássico' } }
```

\$pull

Remove um item de um array por valor

```
{ $pull: { tags: 'rascunho' } }
```

\$addToSet

Adiciona ao array só se não existir

```
{ $addToSet: { tags: 'ação' } }
```

CRUD — Delete (Deletar)

```
// Deletar um documento por filtro
const numRemoved = await db.removeAsync(
  { titulo: 'Matrix' },
  {}
);
```

```
// Deletar por _id
await db.removeAsync({ _id: 'abc123' }, {});
```

```
// Deletar vários (multi: true)
const qtd = await db.removeAsync(
  { ativo: false },
  { multi: true }
);
```

Atenção:

Por padrão o NeDB deleta apenas UM documento. Para vários, passe { multi: true }.

Índices — Acelerando Buscas

Índices tornam buscas por campos específicos muito mais rápidas. Importantes para campos usados frequentemente em filtros.

```
// Índice simples
```

```
await db.ensureIndexAsync({ fieldName: 'titulo' });
```

```
// Índice único (sem duplicatas)
```

```
await db.ensureIndexAsync({
```

```
  fieldName: 'titulo',
```

```
  unique: true,
```

```
});
```

```
// Índice esperso (ignora docs sem o campo)
```

```
await db.ensureIndexAsync({
```

```
  fieldName: 'email',
```

```
  sparse: true,
```

```
});
```

```
// Remover índice
```

```
await db.removeIndexAsync('titulo');
```

Arquitetura do Projeto — MVC com NeDB

projeto-fastify-nedb/

└─ .env

└─ package.json

└─ server.js ← ponto de entrada

└─ data/ ← arquivos .db gerados

 └─ filmes.db

└─ src/

 └─ database/

 └─ index.js ← datastores centralizados

 └─ controllers/

 └─ filmeController.js ← lógica CRUD

 └─ routes/

 └─ filmeRoutes.js ← definição de rotas

Centralizando os Datastores

```
// src/database/index.js
const Datastore = require('@seald-io/nedb');
const path = require('path');

const db = {
  filmes: new Datastore({
    filename: path.join(__dirname, '../..data/filmes.db'),
    autoload: true,
    timestampData: true,
  }),
};

db.filmes.ensureIndexAsync({ fieldName: 'titulo' });

module.exports = db;
```

Importar nos controllers:

```
const db = require('../database');
```

server.js e .env

.env

```
PORT=3000  
NODE_ENV=development
```

server.js

```
require('dotenv').config();  
const fastify = require('fastify')({ logger: true });  
  
fastify.register(require('@fastify/cors'));  
fastify.register(require('./src/routes/filmeRoutes'));  
  
fastify.setErrorHandler((error, request, reply) => {  
  reply.code(error.statusCode || 500).send({  
    sucesso: false,  
    erro: error.message,  
  });  
});
```

Controller — Listar e Criar

```
const db = require('../database');

// GET /filmes
async function listarFilmes(request, reply) {
  const { genero } = request.query;
  const filtro = genero ? { genero } : {};
  const filmes = await db.filmes.findAsync(filtro)
    .sort({ createdAt: -1 });
  return reply.send({ sucesso: true, dados: filmes });
}

// POST /filmes
async function criarFilme(request, reply) {
  const { titulo, diretor, ano, genero, nota } = request.body;
  if (!titulo) {
    return reply.code(400).send({ erro: 'Título obrigatório' });
  }
  const filme = await db.filmes.insertAsync({
    titulo, diretor, ano, genero, nota,
  });
  return reply.code(201).send({ sucesso: true, dados: filme });
}
```

Controller — Buscar por ID

```
// GET /filmes/:id
async function buscarFilme(request, reply) {
  const { id } = request.params;

  const filme = await db.filmes.findOneAsync({ _id: id });

  if (!filme) {
    return reply.code(404).send({
      sucesso: false,
      erro: 'Filme não encontrado',
    });
  }

  return reply.send({ sucesso: true, dados: filme });
}
```

No NeDB o `_id` é uma string simples — não precisa converter para ObjectId como no SQL.

Controller — Atualizar e Deletar

```
// PUT /filmes/:id
async function atualizarFilme(request, reply) {
  const numAffected = await db.filmes.updateAsync(
    { _id: request.params.id },
    { $set: request.body },
    {}
  );
  if (numAffected === 0) {
    return reply.code(404).send({ erro: 'Não encontrado' });
  }
  const atualizado = await db.filmes.findOneAsync({
    _id: request.params.id,
  });
  return reply.send({ sucesso: true, dados: atualizado });
}
```

```
// DELETE /filmes/:id
async function deletarFilme(request, reply) {
  const numRemoved = await db.filmes.removeAsync(
    { _id: request.params.id }
  );
}
```

Arquivo de Rotas — filmeRoutes.js

```
// src/routes/filmeRoutes.js
const {
  listarFilmes, criarFilme,
  buscarFilme, atualizarFilme, deletarFilme,
} = require('../controllers/filmeController');

async function filmeRoutes(fastify) {
  fastify.get('/filmes',    listarFilmes);
  fastify.post('/filmes',   criarFilme);
  fastify.get('/filmes/:id', buscarFilme);
  fastify.put('/filmes/:id', atualizarFilme);
  fastify.delete('/filmes/:id', deletarFilme);
}

module.exports = filmeRoutes;
```

Rotas disponíveis na API:

GET /filmes

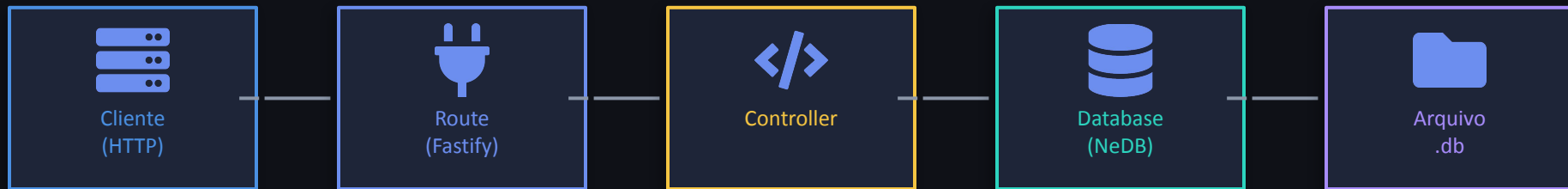
POST /filmes

GET /filmes/:id

PUT /filmes/:id

DELETE /filmes/:id

Fluxo Completo da Requisição



1. Cliente faz a requisição HTTP (ex: GET /filmes)
2. Fastify identifica a rota e chama o handler correto
3. Controller executa a lógica e chama o módulo de banco
4. NeDB faz a busca/escrita no arquivo .db em disco
5. Dado retorna pelo mesmo caminho até a resposta JSON

Boas Práticas com NeDB



Nunca commite a pasta data/

Adicione data/ ao .gitignore. Os arquivos .db contêm dados reais e não devem ir para o repositório.



Valide os dados no controller

O NeDB não tem schema — é sua responsabilidade validar os campos antes de inserir.



Use timestampData: true

Garante que todo documento tenha createdAt e updatedAt gerados automaticamente.



Crie índices para campos de busca

Use ensureIndexAsync() na inicialização. Sem índice, buscas percorrem todos os documentos.



Centralize os datastores

Crie um módulo src/database/index.js e importe de lá. Evita criar múltiplas instâncias do mesmo arquivo.

Referências

- [@seald-io/nedb](https://npmjs.com/package/@seald-io/nedb) → npmjs.com/package/@seald-io/nedb
- [NeDB GitHub](https://github.com/seald-io/nedb) → github.com/seald-io/nedb
- [Fastify Docs](https://fastify.dev) → fastify.dev
- [@fastify/cors](https://npmjs.com/package/@fastify/cors) → npmjs.com/package/@fastify/cors
- [dotenv](https://npmjs.com/package/dotenv) → npmjs.com/package/dotenv